COMPREHENSIVE C++ PROGRAMMING BLUEPRINT · MIDTERM EXAM COMPANION

MIAE 215 Midterm Master Document Cheatsheet

Exact syntax, memory bounds, operator precedence, Gordon table-tracing mechanics & 60% write-a-program recipes.

60% Written Code | 40% Table Tracing | Chapters 1–7 | Zero Garbage Leaks

1. CORE LANGUAGE ARCHITECTURE, PRECEDENCE & CONTROL FLOW

Lectures 1–5 · Foundation Principles

DATA TYPES & NUMERICAL TRAPS — MEMORY / LIMITS

TYPE	BYTES	RANGE & PRECISE BEHAVIOR
`bool`	1	`true`(1), `false`(0); any non-zero is true!
`char`	1	`'0'`=48, `'A'`=65, `'a'`=97. Math: `'c'-'a'=2`
`int`	4	-2.14B to +2.14B. **Integer division truncates!**
`unsigned`	4	0 to 4.29B. Underflow: `0u - 1u == 4294967295`
`double`	8	~15-16 dig. Compare: `fabs(a-b) < 1e-7`

#1 Trap — Truncating Division: `7/2 = 3`, `1/2 = 0`, `1/(i+1) = 0` for $i \geq 1$. Cast *before*: `(double)a / b = 3.5`. Note: `(double)(a / b) = 3.0` (too late!).

Modulo % Rules: Integers only. Result sign strictly follows left operand: `-7 % 3 = -1`; `7 % -3 = 1`; `-7 % -3 = -1`.

PRECEDENCE & EVALUATION ORDER — LOGIC / SHORT-CIRCUIT

- **Strict Precedence:** `() [] x++ x--` → `++x --x ! (type) sizeof` (R-to-L) → `* / %` → `+ -` → `<< >>` → `< <= > >=` → `== !=` → `&&` → `||` → `?:` → `= += -= *=` (R-to-L).
- **Prefix vs Postfix:** `y = ++x;` (increments *x* first, assigns new *x*). `y = x++;` (assigns original *x*, then increments *x*).
- **Chained Relational Trap:** `1 < x < 5` parses as `(1 < x) < 5`. Since `(0 or 1) < 5` is always 1, it is **ALWAYS TRUE**! Must write: `(1 < x && x < 5)`.
- **Assignment in `if`:** `if(x = 5)` assigns 5 to *x* and evaluates true! Always use equality test: `if(x == 5)`.
- **Short-Circuit Protection:** In `A && B`, if *A* is false, *B* is skipped. Safely protects array access: `if(i < n && A[i] > 0)`!

CONTROL FLOW, LOOPS & SEMICOLONS — BRANCHES / LOOP TRIPS

- **Dangling `else`:** Binds to closest unmatched `if` regardless of indentation! Use explicit braces `{ }`.
- **Switch Constraints:** Expression *must* be integral (`int`, `char`, `bool`). Cannot switch on `double`/`float`! Each `case` requires `break;` or execution cascades through next cases!
- **Loop Iteration Trips:**
  - `for(i=0; i<n; i++)` → runs exactly *n* times; final $i = n$.
  - `for(i=0; i<=n; i++)` → runs $n + 1$ times; final $i = n + 1$.
  - `do { ... } while(cond);` → guaranteed ≥ 1 run! Semicolon mandatory!
- **Semicolon Trap:** `for(i=0; i<5; i++); { cout<<i; }` → empty body loops 5 times; then prints `5` once!
- **Jump Keywords:** `break` exits innermost loop/switch; `continue` skips rest of body to next update step.

2. GORDON TRACING PROTOCOL (40%), ARRAY BOUNDARIES & FUNCTIONS

Exam Question 2 Signature · State Verification

DR. GORDON TRACING PROTOCOL (40%) — QUESTION 2 CORE

Step-by-Step Rule: Draw column for each variable + **Output**. Only write when value *changes*. Print strictly when **cout** executes.

Iteration	i	sum	a	B[i]	Output
init	0	0	1	11	–
iter 1	1	11	8	7	**1 8**
iter 2	2	18	14	6	**2 14**
iter 3	3	24	19	5	**3 19**
after loop	3	29	19	–	**29 19 26**
final A[i]	4	–	–	G	Garbage (G)

The Signature Garbage (G) Rule: For array `A[4]`, valid indices are `0..3`. When `for(i=0; i<4; i++)` terminates, `i == 4`! Printing `cout << A[i];` accesses out-of-bounds memory → **Write "Garbage" or "G"** for full marks!

ARRAYS (1D & 2D) & C-STRINGS — MEMORY LAYOUT

- **Partial Initialization:** `int A[5] = {10, 20};` → `A[0]=10, A[1]=20`, and `A[2]=A[3]=A[4]=0` (guaranteed zeroed, *never* garbage!).
- **Total Element Count:** `int n = sizeof(A) / sizeof(A[0]);`
- **2D Matrix Row-Major:** `M[R][C]` has row $i \in [0, R-1]$ and col $j \in [0, C-1]$. Cell address: `*(M[0] + i*C + j)`. When passing to functions, column size **MUST** be constant: `void f(double M[][4], int r)`.
- **C-String Null Terminator:** `char s[20];` **MUST** end with `'\0'` (ASCII 0). Size must be ≥ length + 1! `strlen(s)` counts chars excluding `'\0'`.
- **Stream Reading:** `cin >> s;` stops at whitespace. `cin.getline(s, 20);` reads entire line including spaces up to 19 characters.

FUNCTIONS, REFERENCES & FILE I/O — MODULAR DESIGN

- **Pass-by-Value:** `void f(int x)` → makes local copy; changes inside do NOT modify the caller's variable.
- **Pass-by-Reference:** `void f(int &x)` → receives memory alias; changes **directly modify caller**! Classic swap: `void swap(double &a, double &b){ double t=a; a=b; b=t; }`
- **Arrays to Functions:** Always decay to pointers: `void f(double A[], int n)`. Modifies caller array directly! Array size cannot be known via `sizeof` inside function; always pass *n*.
- **File Streams (`<fstream>`):**
  - Reading: `ifstream fin("data.txt"); if(!fin) return 1;`
  - Reading loop: `while(fin >> val && n < MAX) A[n++] = val; fin.close();`
  - Writing: `ofstream fout("out.txt"); fout << fixed << setprecision(3) << ans; fout.close();`

3. THE 60% WRITE-A-PROGRAM MASTER ALGORITHM BRIDGE (EXAM CRITICAL SITUATIONS)

DR. GORDON HANDWRITTEN CODE SOLUTIONS · TESTED TEMPLATES

1. Dynamic 1D & 2D Array Zero-Leak — new[] / delete[]

```
// 1D Dynamic Array:
int n; cin >> n;
double *A = new double[n];
if (!A) { cout << "Error"; return 1; }
delete[] A; A = nullptr;

// 2D Dynamic Array (R x C):
double **M = new double*[R];
for (int i = 0; i < R; i++) M[i] = new double[C];
for (int i = 0; i < R; i++) delete[] M[i];
delete[] M; M = nullptr;
```

2. Array Equality & Comparison — Tolerance & Break

```
bool match = true;
for (int i = 0; i < n; i++) {
  // Doubles tolerance: fabs(A[i]-B[i]) > 1e-7
  if (A[i] != B[i]) {
    match = false; break;
  }
}
if (match) cout << "Arrays equal";
else cout << "Arrays not equal";
```

3. In-Place Reversal & Merge — No Extra Array

```
// Reverse array A of size n in-place
for (int i = 0; i < n / 2; i++) {
  double temp = A[i];
  A[i] = A[n - 1 - i];
  A[n - 1 - i] = temp;
}
// Concatenate A and B (size n) into C (2n):
for (int i = 0; i < n; i++) {
  C[i] = A[i]; C[i + n] = B[i];
}
```

4. Min/Max & Safe Init — Never Init to 0!

```
// NEVER init max/min to 0 (fails if all < 0!)
double maxV = A[0]; int maxIdx = 0;
for (int i = 1; i < n; i++) {
  if (A[i] > maxV) { maxV = A[i]; maxIdx = i; }
}
// Dot product & Average:
double dot = 0.0;
for (int i = 0; i < n; i++) dot += A[i] * B[i];
double avg = (n > 0) ? (double)dot / n : 0.0;
```

5. C-String Space Stripper — '\0' Re-seal Trap

```
void remove_spaces(char s[]) {
  int j = 0;
  for (int i = 0; s[i] != '\0'; i++) {
    if (s[i] != ' ') s[j++] = s[i];
  }
  s[j] = '\0'; // CRUCIAL: Re-terminate null!
}
// String copy without <cstring>:
for (int i = 0; (dest[i] = src[i]) != '\0'; i++);
```

6. 2D Matrix Sum & Transpose — Nested Loops

```
// Transpose square matrix M[N][N]:
for (int i = 0; i < N; i++) {
  for (int j = i + 1; j < N; j++) {
    double t = M[i][j];
    M[i][j] = M[j][i]; M[j][i] = t;
  }
}
// Row sum: for(j=0;j<C;j++) rsum += M[i][j];
```